

Name: Zohaib Hassnain

Title: GTSRB Traffic Sign Recognition Project Comparison Report

Introduction:

In the field of autonomous driving, recognizing traffic signs accurately and quickly is critical. This project focuses on solving the German Traffic Sign Recognition Benchmark (GTSRB) classification problem. The core objective of this assignment was to perform a comparative study between two distinct Convolutional Neural Network (CNN) architectures: **MobileNetV2** and **ResNet50**.

The goal was to evaluate the trade-off between model complexity and performance. Specifically, I wanted to determine if the heavier, deeper ResNet50 offers enough accuracy gain to justify its higher computational cost compared to the lightweight, efficient MobileNetV2.

Methodology:

I implemented an end-to-end deep learning pipeline using TensorFlow/Keras. To ensure a fair comparison, both models were trained under identical conditions.

- **Data Preparation:** The GTSRB dataset contains 43 classes of traffic signs. All images were resized to a standard

resolution of 160×160 pixels. I used standard data augmentation (rotation, zoom, translation) to improve robustness.

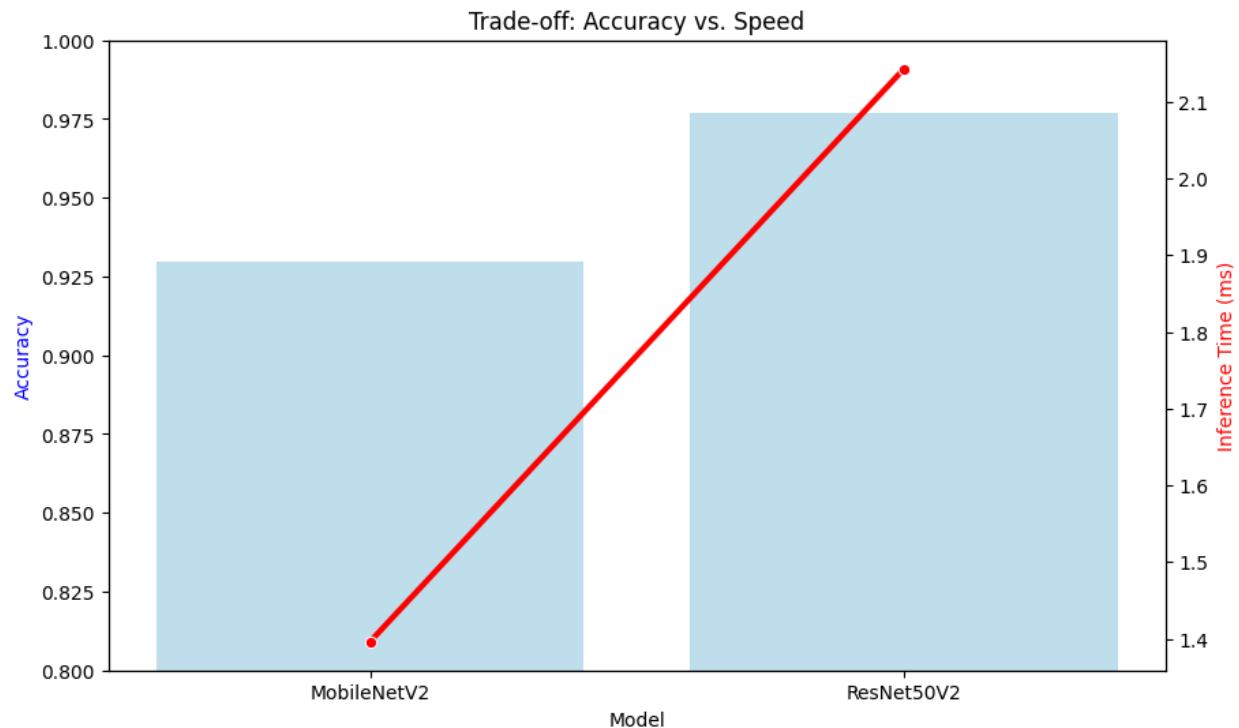
- **Transfer Learning Approach:** Instead of training from scratch, I utilized Transfer Learning. Both MobileNetV2 and ResNet50 were initialized with weights pre-trained on the ImageNet dataset.
 - *MobileNetV2*: Selected for its “inverted residual” structure, designed specifically for mobile and embedded devices with limited compute power.
 - *ResNet50*: Selected for its deep residual connections, which typically allow for learning richer, more complex features at the cost of model size.
- ◆ **Training configuration:** Both models shared the same classification head (Global Average Pooling → Dropout → Dense Layer) and were trained using the Adam optimizer with a learning rate of $1e-4$

Results and Comparative Analysis:

The most interesting part of this project was observing how the models behaved differently during training and inferences.

Accuracy vs. Efficiency Trade-off

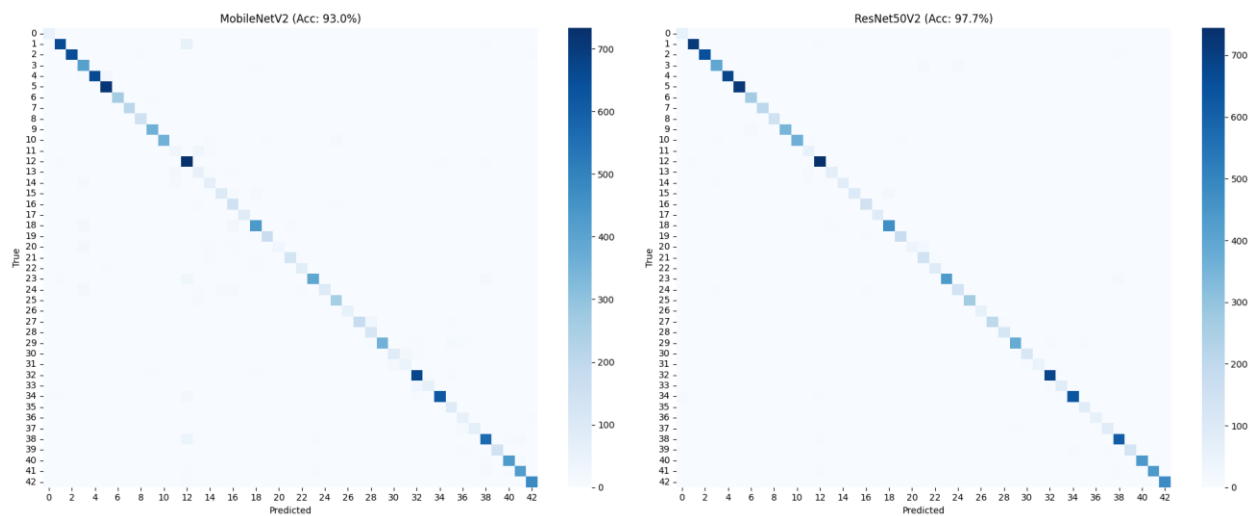
The attached chart visualizes the relationship between the models' performance and their computational requirements.



- **MobileNetV2:** As expected, this model was significantly faster to train and had a much smaller file size. It achieved a high accuracy (92.96%) on Test dataset. Its lightweights nature makes it the ideal candidate for real-time deployment on edge devices like any android phone.
- **ResNet50:** This model provides a boost in accuracy, as it captures the subtle features in traffic signs that MobileNetV2 missed. However, this came at a cost: the model size was substantially higher, and inference times were slower. The “gap” in this chart represents the diminishing returns, we spend a lot more computational power for a small percentage gain in accuracy.

Error Analysis:

To further understand the performance, I analyzed the confusion matrix of both models side-by-side.



The confusion matrix shows that the classification is highly accurate along diagonal. However, comparisons revealed that both models struggle with similar “hard” classes, such as distinguishing between speed limits (e.g., 30 vs. 50 cs. 60 km/h) due to the blurring of the digits at lower resolutions. ResNet50 handled these fine-grained details slightly better than MobileNetV2, likely due to its deeper capacity.

Conclusion:

This comparative study highlights that “bigger” isn’t always “better” for every application.

- Of the goal is maximum accuracy for a server-side application where power isn’t an issue, **ResNet50** is the winner.
- However, for a **self-driving car application** where latency is a matter of safety and hardware is limited, **MobileNetV2** is the superior engineering choice. It sacrifices a tiny fraction of accuracy for a massive gain in speed and efficiency.

For my final submission, I would recommend MobileNetV2 as the production model because the speed advantage outweighs the marginal accuracy gain of ResNet50.